Technical Blocker Mitigation — Defender for Identity

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Technical Blocker Mitigation: Defender for Identity 5
- What you get . . . 5
- What this delivers . . . 5
- The standard deployment model . . . 5
- Boundary . . . 5
- Effort . . . 6

1. Solution Overview — Technical Blocker Mitigation: Defender for Identity 6
- What this solution is . . . 6
- How Defender for Identity signal actually works (so you can find the break) . . . 6
- The standard deployment model (applied to a blocker) . . . 7
- Scope . . . 7
- Out of scope . . . 7
- Connections and authentication used . . . 8

2. Requirements Guide — Technical Blocker Mitigation: Defender for Identity 8
- 2.1 Read-only access to the Defender portal and Graph hunting . . . 8
- 2.2 Local read access on the affected domain controller . . . 8
- 2.3 The affected sensor / domain-controller subset defined . . . 8
- 2.4 Directory Service Account state known . . . 9
- 2.5 Network / NNR reachability knowledge . . . 9
- 2.6 A place to store the diagnostic output and support bundle . . . 9
- Checklist . . . 9

Global Requirements — Defender (category-level) 9
- G1. Microsoft 365 / Entra tenant . . . 10
- G2. Defender licensing . . . 10
- G3. Administrative roles (least privilege) . . . 10
- G4. Onboarding reach to endpoints / identity / workloads . . . 10
- G5. Network access to Defender service endpoints . . . 10
- G6. A pilot ring . . . 10
- G7. Log Analytics workspace (where required) . . . 11
- G8. Response governance . . . 11

3. Implementation Guide — Technical Blocker Mitigation: Defender for Identity 11
- Naming (set once, reuse verbatim) . . . 11
- Step 1 — Assess: pinpoint the blocker (read-only) . . . 11
- Step 2 — Assess: confirm sensor reach (read-only) . . . 12
- Step 3 — Configure: apply the one targeted fix (manual, documented, in a change window) . . . 12

Step 4 — Pilot: re-test the fix on the affected sensor only . . . 13
Step 5 — Validate: confirm the sensor is healthy and signal flows . . . 13
Step 6 — Expand: resume the stalled rollout . . . 13
Step 7 — Escalate (only if it is a service issue, not your config) . . . 13
5. Testing Guide — Technical Blocker Mitigation: Defender for Identity 14
Test 1 — Blocker pinpointed (baseline captured) . . . 14
Test 2 — Sensor healthy . . . 14
Test 3 — DSA working (AD-query signal) . . . 14
Test 4 — Network traffic / NNR . . . 14
Test 5 — Event collection . . . 14
Test 6 — Licensing present . . . 15
Test 7 — Before/after evidence produced . . . 15
Test 8 — Escalation bundle valid (only if escalating) . . . 15
6. Troubleshooting Guide — Technical Blocker Mitigation: Defender for Identity 15
MDI sensor unhealthy . . . 15
DSA missing or under-permissioned . . . 15
Missing network traffic / NNR blind . . . 16
Event-collection gap . . . 16
Licensing / activation blocker . . . 16
Script cannot authenticate / run . . . 16
When to escalate to Microsoft support . . . 16
7. Best Practices — Technical Blocker Mitigation: Defender for Identity 17
Diagnose before you change a domain controller . . . 17
Change one thing, on the affected sensor, in a change window, then verify . . . 17
Keep everything read-only until you deliberately fix . . . 17
Protect the DSA and the sensors . . . 17
Let health and tables warm up . . . 17
Keep licensing and requirement facts current . . . 18
Escalate with evidence, not a description . . . 18
Defer the security decisions to the owners . . . 18
Where to go next . . . 18
8. Deliverables Summary — Technical Blocker Mitigation: Defender for Identity 18
Documentation . . . 18
PowerShell scripts (Graph advanced-hunting KQL + a local DC readiness check — all read-only) . 19
What the buyer receives . . . 19
Outcome . . . 19

Appendix A — PowerShell Scripts 21

Executive Summary — Technical Blocker Mitigation: Defender for Identity

White-label · Defender

A Defender for Identity (MDI) rollout almost always stalls on the same short list of blockers: a **sensor that will not install or reports Unhealthy**, a **Directory Service Account (DSA)** with the wrong permissions, **missing network traffic** so Network Name Resolution (NNR) and lateral-movement detection are blind, or **event-collection gaps** where the required Windows audit events never reach the sensor. Left undiagnosed, MDI looks "deployed" but detects nothing. This solution is a **diagnostic-first playbook**: read-only discovery that pinpoints *which* blocker you actually have, the **targeted, documented manual fix**, and the verification that proves the sensor is healthy and identity signal is flowing — so a stuck rollout gets moving again fast.

This is a **run-it-yourself** package written for an administrator with no prior deep MDI troubleshooting experience. Every script is **read-only** — it reads sensor and DSA state, audit-policy readiness, and identity telemetry; it never changes a sensor, an account, or an audit policy. Any fix is a **documented manual step** you apply deliberately, on the affected sensor/domain-controller subset first.

What you get

- A plain-language **Solution Overview** of the MDI signal chain — activation, sensor health, the DSA, network traffic / NNR, and event collection — and where each one breaks.
- A **Requirements Guide** with verify / implement-if-missing for the diagnostic-specific prerequisites.
- An **Implementation Guide** that walks the diagnostic loop end to end — pinpoint the blocker, apply the targeted fix, re-test the affected sensor, confirm healthy, resume the rollout.
- Five **read-only PowerShell scripts** (Microsoft Graph advanced-hunting KQL + a local DC audit-policy/NNR readiness check) for discovery, audit-policy readiness, blocker validation, a status report, and a support-ready data bundle.
- **Testing** and **Troubleshooting** guides that map every common symptom to its cause and fix.

What this delivers

A previously stuck Defender for Identity rollout **moving again** — the blocker diagnosed, the targeted fix applied and verified on the affected sensor, and, if the root cause is a Microsoft service issue rather than your configuration, a **support-ready diagnostic bundle** and clear escalation criteria.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — assess with read-only discovery to pinpoint the blocker, apply the targeted fix, pilot the fix on the affected sensor/DC, validate the sensor is healthy and identity signal flows, then expand by resuming the stalled rollout to the remaining domain controllers.

Boundary

This is a **diagnostic and troubleshooting** solution. It gets a *stalled* MDI deployment unstuck; it does **not** replace the full identity-onboarding engagement (see the adjacent *Defender XDR Deployment & Unified Onboarding* solution). All scripts are **read-only**. Identity response actions (disable user, force password reset, confirm compromised) stay **approval-gated** and are the SOC's call; incident-response and licensing determinations remain the customer's functions — this guide provides the technical diagnosis and the documented fix, not the decision.

Effort

A competent administrator can run discovery, identify the blocker, apply the documented fix, and verify sensor health for a single blocker class in a focused session. Rolling the fix and sensors out to the remaining domain controllers scales with DC count and change windows.

1. Solution Overview — Technical Blocker Mitigation: Defender for Identity

What this solution is

This is a **diagnostic and troubleshooting playbook** for a **stalled Microsoft Defender for Identity (MDI) rollout**. It assumes you have started deploying sensors and hit a wall — a sensor that will not go healthy, or one that is "healthy" but detects nothing. Its job is to answer three questions quickly and in order:

1. **Which blocker do I actually have?** (read-only discovery of sensor, DSA, traffic, and event state — not guesswork)
2. **What is the exact, documented fix?** (a targeted manual step, applied deliberately)
3. **Is the sensor healthy and is identity signal flowing?** (verification before you resume)

It targets the four blocker classes that stall almost every MDI deployment:

• **Sensor install / health failures** — the sensor will not install, or shows **Unhealthy / Not configured / disconnected** in the portal (service not running, outbound reach to the MDI service blocked, unsupported OS, or an out-of-date sensor).

• **DSA permission issues** — the **Directory Service Account** (gMSA recommended) is missing or under-permissioned, so the sensor cannot query AD for the context several detections need.

• **Missing network traffic** — the sensor is not seeing the domain controller's traffic, so **Network Name Resolution (NNR)** and network-based detections are blind (NNR ports blocked, or the wrong adapter).

• **Event-collection gaps** — the required **Windows Security audit events** (e.g. the account-logon / directory-service audit categories) are not being generated or forwarded, so event-based detections are silent.

How Defender for Identity signal actually works (so you can find the break)

MDI has a signal chain, and a blocker is always a break in one specific link:

1. **Licensing/activation** — the tenant has an MDI plan and the MDI workspace is activated in `security.microsoft.com` → **Settings** → **Identities**. *Verify current plan requirements in your

tenant.*

2. **Sensor installed & healthy** — the sensor is installed on the domain controller (and AD FS / AD CS where present), the service runs, and it has **outbound reach** to the MDI service so it reports **Running / Healthy**.

3. **DSA configured** — a read-only **Directory Service Account** is registered so the sensor can enrich detections with AD queries.

4. **Network traffic seen** — the sensor captures the DC's network traffic and performs **NNR** (needs the relevant ports open to endpoints) to resolve IPs to devices.

5. **Events collected** — the required **advanced audit policy** is enabled so the sensor receives the Windows Security events its detections depend on.

6. **Verification** — sensor health shows **Healthy** in the portal, and advanced-hunting tables (`IdentityInfo`, `IdentityLogonEvents`, `IdentityDirectoryEvents`, `IdentityQueryEvents`) stream.

Find the earliest broken link and you have found the blocker. Discovery reports the state of each link so you are not guessing.

The standard deployment model (applied to a blocker)

> ***Assess*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand. Assess*** *— run read-only discovery and the audit-policy readiness check to pinpoint the earliest broken link.* ***Configure*** *— apply the single documented fix for that blocker (a manual step in a change window, not a script action).* ***Pilot*** *— re-test on the* ***affected sensor/DC*** *only.* ***Validate*** *— confirm the sensor is healthy and identity signal flows.* ***Expand*** *— resume the stalled rollout to the remaining domain controllers.*

Scope

• Diagnosing and clearing the four blocker classes above for a stalled MDI rollout.

• Read-only discovery, audit-policy / NNR readiness checking, validation, status reporting, and a support-ready bundle.

• Documented manual fixes for each blocker, applied affected-sensor-first, in a change window.

• Clear escalation criteria and the exact data to collect when the root cause is a Microsoft service issue.

Out of scope

• **First-time identity onboarding of a greenfield tenant** — that is part of the *Defender XDR Deployment & Unified Onboarding* solution; this one unblocks a rollout already in progress.

• **Identity response and incident handling** — disable user, force password reset, and confirm-compromised actions stay **approval-gated** and are the SOC's decision.

• **Broader identity hardening** (Conditional Access, PIM, passwordless) — see the adjacent Entra solutions.

• **Licensing determinations** — the guide tells you *which* plan the feature needs and how to verify it; assigning it is the customer's licensing function.

Connections and authentication used

• The **unified Defender portal** (`security.microsoft.com`) — **Settings** → **Identities** → **Sensors / Directory Service accounts**, sensor health.

• **Microsoft Graph advanced-hunting API** (via `Microsoft.Graph`) for read-only `Identity*` table validation.

• **A local, read-only check on the domain controller** — `auditpol` and configuration reads to confirm the advanced audit policy and NNR readiness the sensor needs (no changes made).

• Least-privilege roles: **Security Reader** for the cloud diagnostics; local **read** access on the DC for the audit-policy check; **Security Administrator** (and a change window on the DC) only to apply a fix.

2. Requirements Guide — Technical Blocker Mitigation: Defender for Identity

Verify **every** requirement before diagnosing. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant, Defender licensing, admin roles, onboarding reach, network access, pilot ring, workspace, response governance) are in `global-requirements-defender.md` — do not repeat them; the items below are **specific to running the MDI blocker diagnostics**. Objects you create are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

2.1 Read-only access to the Defender portal and Graph hunting

Description: Security Reader (or higher) in the Defender portal and consent to the read-only advanced-hunting scope (`ThreatHunting.Read.All`). **Why:** The cloud diagnostics read sensor health context and the `Identity*` hunting tables — all read-only — to pinpoint the blocker. **Verify:** Sign in to `security.microsoft.com`; confirm **Settings** → **Identities** → **Sensors** and **Advanced hunting** load. `TBMMDI-Discovery.ps1` reports the roles/scopes it resolved. **Implement-if-missing:** Assign **Security Reader**; grant admin consent to `ThreatHunting.Read.All`.

2.2 Local read access on the affected domain controller

Description: The ability to run a **read-only** local check (as a local admin / audit reader) on the affected DC to inspect the advanced audit policy and NNR readiness. **Why:** Event-collection and NNR blockers are only confirmable on the DC itself — `auditpol` and local configuration reads show whether the sensor is even being fed the events it needs. **Verify:** You can run `TBMMDI-AuditPolicy.ps1` on the affected DC and it reports the audit subcategory states. **Implement-if-missing:** Arrange local read access on the DC (the script makes no changes); coordinate with the AD team.

2.3 The affected sensor / domain-controller subset defined

Description: A named subset of the sensors/DCs exhibiting the blocker — the ones you will re-test the fix against before resuming. Name it `MDI Blocker Triage – Affected Sensors – Diagnose`. **Why:** Fixes are validated on the affected sensor first (per `global-requirements-defender.md` G6) in a change window before touching every DC. **Verify:** The list of affected DCs/sensors exists;

`TBMMDI-Validate.ps1` checks identity signal for them. **Implement-if-missing:** Document the affected DCs/sensors as `MDI Blocker Triage – Affected Sensors – Diagnose`.

2.4 Directory Service Account state known

Description: Know whether a read-only **DSA** (gMSA recommended) is configured under **Settings** → **Identities** → **Directory Service accounts**, and with what account. **Why:** A missing or under-permissioned DSA is a top blocker for the AD-context detections; you must know its state to diagnose it. **Verify:** `security.microsoft.com` → **Settings** → **Identities** → **Directory Service accounts**; `TBMMDI-Discovery.ps1` notes whether identity-query signal appears (a proxy for a working DSA). *Verify current MDI DSA requirements in your tenant.* **Implement-if-missing:** Prepare the gMSA and its read permissions per current MDI guidance for the fix step.

2.5 Network / NNR reachability knowledge

Description: Know the sensor's outbound reach to the MDI service and the **NNR** ports the sensor uses to resolve endpoints from the DC. **Why:** "Missing network traffic" blockers come from blocked NNR ports or blocked outbound service reach; you must be able to check both. **Verify:** `TBMMDI-AuditPolicy.ps1` reports NNR-relevant local readiness; the sensor's own connectivity test (in the sensor UI/logs) confirms outbound reach. *Verify the current MDI service endpoints and NNR ports.* **Implement-if-missing:** Obtain the current MDI endpoint/NNR port list and work with networking to test them.

2.6 A place to store the diagnostic output and support bundle

Description: A secure location for the JSON snapshots the scripts write and for the support bundle `MDI Support Bundle – <case> – Escalate`. **Why:** The before/after snapshots prove the sensor became healthy, and the bundle is what you attach to a Microsoft support case. **Verify:** A working folder exists with least-privilege access; `TBMMDI-Report.ps1` writes there. **Implement-if-missing:** Create a restricted working folder; do not store DC names/tenant identifiers in shared/unsecured locations.

Checklist

- Security Reader + `ThreatHunting.Read.All` — `TBMMDI-Discovery.ps1`
- Local read access on the affected DC — `TBMMDI-AuditPolicy.ps1`
- Affected sensor/DC subset `MDI Blocker Triage – Affected Sensors – Diagnose` documented
- DSA (gMSA) state known
- Network / NNR reachability checkable (service endpoints + NNR ports)
- Secure working folder for snapshots and the support bundle

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying

any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Technical Blocker Mitigation: Defender for Identity

Follow these steps **in order**. This is a **diagnostic loop**, not a build: you assess to pinpoint the blocker, apply one targeted documented fix (in a change window, since it touches a domain controller), re-test the affected sensor, and only then resume the rollout. All scripts are **read-only**; every *fix* is a manual step you apply deliberately. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model:*** *Assess (pinpoint) → Configure (fix) → Pilot (re-test affected sensor) → Validate (confirm healthy + signal) → Expand (resume rollout). Never change a DC's audit policy or account outside a change window, and keep identity response actions approval-gated throughout.*

Naming (set once, reuse verbatim)

- Affected sensor/DC subset: `MDI Blocker Triage – Affected Sensors – Diagnose`
- Advanced-hunting saved query (sensor/telemetry check): `MDI Sensor Blocker Check – Tenant – Diagnose`
- Support bundle output: `MDI Support Bundle – <case> – Escalate`
- Discovery snapshot file: `MDI-blocker-discovery.json`

Step 1 — Assess: pinpoint the blocker (read-only)

1. Sign in to `security.microsoft.com` as **Security Reader** and open **Settings** → **Identities** → **Sensors** to read current sensor health.

2. Run `TBMMDI-Discovery.ps1`. It captures the cloud side of the signal chain: licensed MDI plan, whether identity signal is arriving (`IdentityInfo` / `IdentityLogonEvents` presence), and identity-query signal as a proxy for a working DSA. It records the current state to `MDI-blocker-discovery.json`.

3. On a **sample affected DC** from `MDI Blocker Triage – Affected Sensors – Diagnose`, run `TBMMDI-AuditPolicy.ps1` (read-only) to capture the advanced-audit-policy and NNR readiness the sensor depends on.

4. Read both outputs top-down against the signal chain (Overview §"How signal actually works") and identify the **earliest broken link**:

• No plan / workspace not activated → **licensing/activation blocker**

• Sensor Unhealthy/disconnected → **sensor install/health blocker**

• No identity-query signal → **DSA blocker**

• NNR ports blocked / no traffic → **network-traffic blocker**

• Audit subcategories not enabled → **event-collection blocker**

Step 2 — Assess: confirm sensor reach (read-only)

1. In the sensor's own UI/logs on the affected DC, run its built-in connectivity test to confirm **outbound reach** to the MDI service. *Verify the current MDI service endpoints.*

2. Note whether the failure is service-reach (sensor link) or NNR-port/traffic (network link) — they have different fixes.

Step 3 — Configure: apply the one targeted fix (manual, documented, in a change window)

Apply **only** the fix for the blocker you identified. Each is a deliberate manual step — the scripts never make these changes.

• **Licensing / activation:** Assign the MDI plan (**Billing** → **Licenses**) and activate the MDI workspace in **Settings** → **Identities**. *Verify the exact plan the feature needs in your tenant.*

• **Sensor install / health:** On the affected DC, confirm a **supported OS** and resources, allow the sensor's outbound **service endpoints** at the proxy/firewall (per `global-requirements-defender.md` G5), update to the current sensor version, and reinstall the sensor with a fresh access key if it is corrupt. Confirm the sensor service is running.

• **DSA permissions:** Create/register the read-only **DSA** (gMSA recommended) under **Settings** → **Identities** → **Directory Service accounts** with the current required permissions. *Verify current DSA requirements.*

• **Network traffic / NNR:** Open the required **NNR ports** from the DC to endpoints, and confirm the sensor binds the correct network adapter so it sees the DC's traffic. *Verify current NNR port requirements.*

• **Event-collection gaps:** Enable the required **advanced audit policy** subcategories on the DC (per current MDI guidance) so the sensor receives the needed Windows Security events; for non-DC sensors, configure the documented event forwarding.

Step 4 — Pilot: re-test the fix on the affected sensor only

1. Apply the fix to the sensor/DC in `MDI Blocker Triage – Affected Sensors – Diagnose` only — not every DC.

2. Give the sensor time to reconnect and start reporting (health and tables populate over minutes to hours).

3. Move to validation before touching any other DC.

Step 5 — Validate: confirm the sensor is healthy and signal flows

1. In **Settings** → **Identities** → **Sensors**, confirm the affected sensor shows **Running / Healthy**.

2. Run `TBMMDI-Validate.ps1`. It runs the saved query `MDI Sensor Blocker Check – Tenant – Diagnose` against `IdentityLogonEvents` / `IdentityDirectoryEvents` / `IdentityQueryEvents` and returns pass/fail for identity signal.

3. Re-run `TBMMDI-AuditPolicy.ps1` on the DC to confirm the audit subcategories are now enabled.

4. Run `TBMMDI-Report.ps1` for the before/after evidence.

5. Complete the **Testing Guide** for the blocker class before expanding.

Step 6 — Expand: resume the stalled rollout

1. With the affected sensor healthy and streaming, roll the fix and remaining sensors out to the other domain controllers in change windows until all are healthy.

2. Re-run `TBMMDI-Validate.ps1` after each DC and compare with `TBMMDI-Report.ps1`.

3. If a DC re-introduces the symptom, return to Step 1 for that DC — audit policy or NNR can differ per site.

Step 7 — Escalate (only if it is a service issue, not your config)

1. If the sensor is on a supported OS, has service reach, the DSA and audit policy are correct, and it still will not go healthy or stream, it may be a Microsoft service issue.

2. Run `TBMMDI-SupportBundle.ps1` to assemble `MDI Support Bundle – <case> – Escalate` — the discovery snapshot, the audit-policy readiness output, the affected DC list, and timestamps — and run the **sensor's own log-collection** on the DC to add its diagnostics.

3. Open a Microsoft support case, attach the bundle, and reference the escalation criteria in the Troubleshooting Guide. *Incident-response decisions remain the SOC's; this is a service-support escalation.*

Order of operations

`Assess (discovery + audit-policy readiness) → identify earliest broken link → apply the one targeted fix in a change window → re-test the affected sensor → validate healthy + signal → resume rollout → escalate with a bundle only if it is a service issue`

5. Testing Guide — Technical Blocker Mitigation: Defender for Identity

Run these checks after applying a fix on the affected sensor (`MDI Blocker Triage – Affected Sensors – Diagnose`) and again after each additional domain controller. Each test has a **pass/fail gate**; do not resume the wider rollout until the affected sensor passes the test for its blocker class. Objects are referenced by the exact names from the Implementation Guide.

Test 1 — Blocker pinpointed (baseline captured)

Procedure: Run `TBMMDI-Discovery.ps1` and confirm `MDI-blocker-discovery.json` has the plan and identity-signal presence populated; run `TBMMDI-AuditPolicy.ps1` on the affected DC and confirm subcategory states are captured. **Pass:** A single earliest broken link is identifiable; the affected DCs are in the triage subset. **Fail:** Snapshot empty or access-denied — fix access (Requirements 2.1/2.2) and re-run.

Test 2 — Sensor healthy

Procedure: In `security.microsoft.com` → **Settings** → **Identities** → **Sensors**, confirm the affected sensor shows **Running / Healthy**; run the sensor's own connectivity test. **Pass:** The sensor is Running / Healthy with service reach. **Fail:** Sensor Unhealthy / disconnected — see Troubleshooting *MDI sensor unhealthy*.

Test 3 — DSA working (AD-query signal)

Procedure: Run `TBMMDI-Validate.ps1` and confirm `IdentityQueryEvents` returns recent rows for the affected DCs. **Pass:** AD-query signal is present, indicating a working DSA. **Fail:** No query signal — see Troubleshooting *DSA missing or under-permissioned*.

Test 4 — Network traffic / NNR

Procedure: Run `TBMMDI-AuditPolicy.ps1` (NNR readiness section) and confirm NNR-relevant ports/config are ready; confirm the sensor resolves endpoints (no NNR warnings in the sensor UI). **Pass:** NNR is functioning and the sensor sees DC traffic. **Fail:** NNR warnings / no traffic — see Troubleshooting *Missing network traffic / NNR blind*.

Test 5 — Event collection

Procedure: Run `TBMMDI-AuditPolicy.ps1` and confirm the required advanced-audit subcategories are **enabled**; run `TBMMDI-Validate.ps1` and confirm `IdentityLogonEvents` / `IdentityDirectoryEvents` return recent rows. **Pass:** Required audit subcategories are enabled and event-based identity signal is flowing. **Fail:** Subcategories off or no event signal — see Troubleshooting *Event-collection gap*.

Test 6 — Licensing present

Procedure: Run `TBMMDI-Discovery.ps1` and confirm the MDI plan is reported as licensed and the workspace activated. **Pass:** The required MDI plan is present and activated. *Verify current plan requirements in your tenant.* **Fail:** Plan missing / not activated — see Troubleshooting *Licensing / activation blocker*.

Test 7 — Before/after evidence produced

Procedure: Run `TBMMDI-Report.ps1` and confirm it emits a before/after comparison from the Step 1 snapshot versus current state. **Pass:** The report shows the blocker's link moved from broken to cleared. **Fail:** No change captured — re-run discovery to refresh the baseline.

Test 8 — Escalation bundle valid (only if escalating)

Procedure: If the sensor still will not go healthy or stream after a correct fix, run `TBMMDI-SupportBundle.ps1` and confirm `MDI Support Bundle – <case> – Escalate` contains the discovery snapshot, audit-policy readiness output, affected DC list, and timestamps. **Pass:** A complete, attachable bundle exists (plus the sensor's own log-collection output). **Fail:** Bundle incomplete — re-run discovery / audit-policy first, then rebuild.

Exit gate

The affected sensor passes the tests for its blocker class **before** you resume the wider rollout (Implementation Step 6). Re-run Tests 2–5 and 7 after each additional domain controller and compare against the saved baseline.

6. Troubleshooting Guide — Technical Blocker Mitigation: Defender for Identity

The four blocker classes in **symptom** → **cause** → **fix** form, plus escalation. Run `TBMMDI-Discovery.ps1` and `TBMMDI-AuditPolicy.ps1` (on the affected DC) first — most causes surface there. This *is* the troubleshooting solution, so this guide is the core reference.

MDI sensor unhealthy

Symptom: The sensor on a DC in `MDI Blocker Triage – Affected Sensors – Diagnose` shows **Unhealthy**, **Not configured**, or **disconnected** in **Settings** → **Identities** → **Sensors**. **Cause:** Blocked outbound reach to the MDI service, an out-of-date or corrupt sensor, an unsupported OS, or the sensor service stopped. **Fix:** Run the sensor's built-in connectivity test; allow the MDI **service endpoints** at the proxy/firewall (per `global-requirements-defender.md` G5 — *verify the current endpoint list*). Confirm a supported OS and resources, update the sensor to the current version, and reinstall with a fresh access key if corrupt. Confirm the sensor service is running. Re-check health and re-run `TBMMDI-Validate.ps1`.

DSA missing or under-permissioned

Symptom: The sensor is healthy, but AD-context detections are limited and `IdentityQueryEvents` returns nothing. **Cause:** No **Directory Service Account** is registered, or it lacks the read permissions MDI needs. **Fix:** Create/register a read-only **DSA** (gMSA recommended) under **Settings** → **Identities** → **Directory Service accounts** with the current required permissions (*verify current DSA requirements*). Allow time, then re-run `TBMMDI-Validate.ps1` and confirm query signal.

Missing network traffic / NNR blind

Symptom: The sensor is healthy but network-based detections are silent; the sensor UI shows **NNR** resolution warnings. **Cause:** The **NNR ports** from the DC to endpoints are blocked, or the sensor is bound to the wrong network adapter and does not see the DC's traffic. **Fix:** Open the required **NNR ports** (*verify current NNR port requirements*); confirm the sensor binds the adapter carrying the DC's traffic. Re-run `TBMMDI-AuditPolicy.ps1` (NNR readiness) and confirm the sensor resolves endpoints.

Event-collection gap

Symptom: `IdentityLogonEvents` / `IdentityDirectoryEvents` return nothing even though the sensor is healthy. **Cause:** The required **advanced audit policy** subcategories are not enabled on the DC, so the sensor never receives the Windows Security events its detections depend on (or, for non-DC sensors, event forwarding is not configured). **Fix:** Enable the required advanced-audit subcategories on the DC per current MDI guidance (`TBMMDI-AuditPolicy.ps1` shows which are missing); configure the documented event forwarding for non-DC sensors. Allow time, then re-run `TBMMDI-Validate.ps1`.

Licensing / activation blocker

Symptom: MDI options are missing, or the Identities workspace is not activated. **Cause:** No MDI plan assigned, or first-time workspace activation never completed. **Fix:** Assign the plan (**Billing** → **Licenses**) and activate the workspace in **Settings** → **Identities**. *Verify the exact plan the feature needs — availability changes; confirm in your tenant.* Re-run `TBMMDI-Discovery.ps1`.

Script cannot authenticate / run

Symptom: `TBMMDI-Discovery.ps1` / `TBMMDI-Validate.ps1` fails on connect; `TBMMDI-AuditPolicy.ps1` cannot read the policy. **Cause:** The Microsoft Graph modules are missing, `ThreatHunting.Read.All` consent was not granted, the account lacks Security Reader, or the audit script was not run with local read rights on the DC. **Fix:** Let the cloud scripts install the modules; sign in with `Connect-MgGraph` and consent to the read scope; confirm Security Reader. Run `TBMMDI-AuditPolicy.ps1` locally on the DC with sufficient rights. Re-run.

When to escalate to Microsoft support

Escalate — and attach `MDI Support Bundle – <case> – Escalate` — when **all** of the following hold:

- The sensor is on a supported OS with confirmed outbound service reach.
- The DSA is registered with the required permissions.
- The required audit subcategories are enabled and NNR ports are open.

• The correct documented fix was applied and warm-up time elapsed, yet the sensor still will not go healthy or stream.

In that case the root cause is likely a **service-side issue**, not your configuration. Run `TBMMDI-SupportBundle.ps1`, run the **sensor's own log-collection** on the DC, open the case, and attach both. *Incident-response decisions remain the SOC's; this is a service-support escalation.*

7. Best Practices — Technical Blocker Mitigation: Defender for Identity

Field-proven guidance for diagnosing a stalled MDI rollout cleanly, fixing the *right* thing, and treating domain controllers with the care they deserve.

Diagnose before you change a domain controller

• Run `TBMMDI-Discovery.ps1` and `TBMMDI-AuditPolicy.ps1` **first**. Read the signal chain top-down; the earliest broken link is the blocker. A DC is security-critical infrastructure — do not change audit policy or accounts on a hunch.

• A "healthy but detects nothing" sensor is almost always a **DSA, NNR, or event-collection** gap, not a broken sensor — confirm which link before reinstalling anything.

Change one thing, on the affected sensor, in a change window, then verify

• Apply exactly **one** documented fix, to the DC in `MDI Blocker Triage – Affected Sensors – Diagnose`, inside a change window, then run `TBMMDI-Validate.ps1`. One change at a time keeps cause and effect clear.

• Never resume the DC-wide rollout until the affected sensor passes its test gate.

Keep everything read-only until you deliberately fix

• Every script here is read-only, including the on-DC audit check. Treat the *fix* (audit policy, DSA, ports, sensor reinstall) as a separate, conscious change-window action.

• Never loosen an existing audit or account control beyond what MDI documents — over-permissioning the DSA is a security risk.

Protect the DSA and the sensors

• Use a read-only **gMSA** for the DSA per current guidance; do not grant more than MDI needs. Keep sensors patched to the current version and monitor their health on a cadence — a silent sensor blinds MDI.

Let health and tables warm up

• Sensor health, NNR, and the `Identity*` hunting tables populate over minutes to hours after a fix. Give it time before calling the fix a failure, then re-validate.

Keep licensing and requirement facts current

• MDI is **plan-gated**, and DSA requirements, NNR ports, and supported sensor OS versions **change**. Before concluding "it's broken," *verify the current requirements and service endpoints in your tenant.*

Escalate with evidence, not a description

• If the sensor survives a correct fix, attach `MDI Support Bundle – <case> – Escalate` plus the sensor's own log-collection output to the case. A complete bundle resolves cases far faster than a written symptom.

Defer the security decisions to the owners

• Disable-user, force-password-reset, and confirm-compromised actions are **the SOC/security team's** decisions, kept approval-gated. This solution diagnoses and clears the *deployment* blocker; the response posture stays with the customer.

Where to go next

Once the rollout is unblocked, natural next steps in the catalog:

• **Defender XDR Deployment & Unified Onboarding** to correlate identity alerts with endpoint, email, and cloud-app signal in one portal.

• **Entra ID Conditional Access / PIM / passwordless** solutions to harden the identity layer MDI protects.

• The **Sentinel** SOC/XDR integration to stream MDI signal into a SIEM for long-term hunting.

8. Deliverables Summary — Technical Blocker Mitigation: Defender for Identity

Documentation

Document	Purpose
Executive Summary	The blockers, the diagnostic-first approach, the outcome, the boundary, effort
1. Solution Overview	The four MDI blocker classes, the signal chain (sensor → DSA → traffic/NNR → events), where each breaks
2. Requirements Guide	Diagnostic-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once
3. Implementation Guide	The diagnostic loop — assess, fix in a change window, re-test the affected sensor, validate, resume, escalate
5. Testing Guide	Eight pass/fail tests, one per link in the signal chain

Document	Purpose
6. Troubleshooting Guide	Every blocker class in symptom → cause → fix, plus escalation criteria
7. Best Practices	Diagnose-first discipline, DC care, and where to go next

PowerShell scripts (Graph advanced-hunting KQL + a local DC readiness check — all read-only)

Script	What it does
`TBMMDI-Discovery.ps1`	Read-only cloud-side snapshot — licensed MDI plan, identity-signal presence (`IdentityInfo` / `IdentityLogonEvents`), and identity-query signal as a proxy for a working DSA
`TBMMDI-AuditPolicy.ps1`	Read-only local check on the affected DC — reports the advanced-audit-policy subcategory states and NNR readiness the sensor depends on (makes no changes)
`TBMMDI-Validate.ps1`	Read-only pass/fail for the affected sensor — runs the saved hunting query against `IdentityLogonEvents` / `IdentityDirectoryEvents` / `IdentityQueryEvents` to confirm signal flows
`TBMMDI-Report.ps1`	Read-only before/after status report from the discovery baseline versus current state — the evidence the sensor became healthy
`TBMMDI-SupportBundle.ps1`	Read-only assembly of the support-ready bundle (discovery snapshot, audit-policy readiness output, affected DC list, timestamps) for a Microsoft support case

What the buyer receives

- This solution as a **Word** document and **PDF**.
- All five **read-only PowerShell scripts** (with embedded KQL) in a single ZIP.
- A repeatable diagnostic process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — pinpoint the blocker, apply one documented fix in a change window, re-test the affected sensor, confirm healthy, resume the rollout.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A previously stuck Defender for Identity rollout moving again — the blocker diagnosed against the signal chain, the targeted documented fix applied and verified on the affected sensor, and a support-ready

bundle ready if the root cause turns out to be a Microsoft service issue.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

TBMMDI-AuditPolicy.ps1

Purpose: Run ON an affected domain controller from 'MDI Blocker Triage - Affected Sensors - Diagnose'. Event-collection and network-traffic blockers are only confirmable on the DC. This script reads (read-only) the advanced audit policy subcategory states MDI needs and probes NNR-relevant local readiness, then reports which are missing. It makes NO changes to audit policy, accounts, or network config. HOW TO RUN Run locally on the DC in an elevated PowerShell (read rights are enough): powershell -ExecutionPolicy Bypass -File .\TBMMDI-AuditPolicy.ps1 EXPECTED RESULTS A table of the required audit subcategories with Enabled/Disabled state and an NNR readiness summary, written to MDI-auditpolicy-<hostname>.json. Disabled subcategories point at the event-collection fix; NNR gaps point at the network-traffic fix. .NOTES White-label. Read-only — uses 'auditpol /get' and read-only probes only; changes nothing. The exact required subcategories and NNR ports EVOLVE — verify the current MDI requirements and treat this as a readiness indicator, not an authority. > [CmdletBinding()] param([string]$OutputPath = ".\MDI-auditpolicy-$env:COMPUTERNAME.json")

How to run: Run locally on the DC in an elevated PowerShell (read rights are enough): powershell -ExecutionPolicy Bypass -File .\TBMMDI-AuditPolicy.ps1 EXPECTED RESULTS A table of the required audit subcategories with Enabled/Disabled state and an NNR readiness summary, written to MDI-auditpolicy-<hostname>.json. Disabled subcategories point at the event-collection fix; NNR gaps point at the network-traffic fix. .NOTES White-label. Read-only — uses 'auditpol /get' and read-only probes only; changes nothing. The exact required subcategories and NNR ports EVOLVE — verify the current MDI requirements and treat this as a readiness indicator, not an authority. > [CmdletBinding()] param([string]$OutputPath = ".\MDI-auditpolicy-$env:COMPUTERNAME.json")

TBMMDI-Discovery.ps1

Purpose: Read-only discovery for the "Technical Blocker Mitigation — Defender for Identity" solution. Captures the cloud side of the MDI signal chain: the licensed MDI plan (indicative), whether identity signal is arriving (IdentityInfo / IdentityLogonEvents presence), and identity-query signal (IdentityQueryEvents) as a proxy for a working Directory Service Account. Pair it with TBMMDI-AuditPolicy.ps1 run ON the affected DC for the on-prem links. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMDI-Discovery.ps1 Requires the Microsoft.Graph modules. Connects read-only with Connect-MgGraph and prompts for consent to read scopes only. EXPECTED RESULTS A JSON snapshot (MDI-blocker-discovery.json) plus a console summary. Read it with the DC audit output top-down against the signal chain; the earliest broken link is the blocker. .NOTES White-label. Read-only. Reports on state/telemetry only; changes nothing. Sensor health and DSA config live in security.microsoft.com -> Settings -> Identities (portal). Licensing facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDI-blocker-discovery.json", [int]$LookbackHours = 24)

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMDI-Discovery.ps1 Requires the Microsoft.Graph modules. Connects read-only with Connect-MgGraph and prompts for consent to read scopes only. EXPECTED RESULTS A JSON snapshot (MDI-blocker-discovery.json) plus a console summary. Read it with the DC audit output top-down against the signal chain; the earliest broken link is the blocker. .NOTES White-label. Read-only. Reports on state/telemetry only; changes nothing. Sensor health and DSA config live in security.microsoft.com -> Settings -> Identities (portal). Licensing facts are

indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDI-blocker-discovery.json", [int]$LookbackHours = 24)

TBMMDI-Report.ps1

Purpose: Read-only reporting for the "Technical Blocker Mitigation — Defender for Identity" solution. Compares the discovery baseline (MDI-blocker-discovery.json) with a fresh live read of the same identity-signal links and prints a before/after summary — the evidence the sensor became healthy and signal is flowing. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMDI-Report.ps1 -Baseline .\MDI-blocker-discovery.json Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A table of each link's baseline vs current row-count and an overall CLEARED / STILL-BLOCKED verdict, written to MDI-blocker-report.json. .NOTES White-label. Read-only. Reads telemetry only; changes nothing. > [CmdletBinding()] param([string]$Baseline = ".\MDI-blocker-discovery.json", [string]$OutputPath = ".\MDI-blocker-report.json", [int]$LookbackHours = 24)

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMDI-Report.ps1 -Baseline .\MDI-blocker-discovery.json Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A table of each link's baseline vs current row-count and an overall CLEARED / STILL-BLOCKED verdict, written to MDI-blocker-report.json. .NOTES White-label. Read-only. Reads telemetry only; changes nothing. > [CmdletBinding()] param([string]$Baseline = ".\MDI-blocker-discovery.json", [string]$OutputPath = ".\MDI-blocker-report.json", [int]$LookbackHours = 24)

TBMMDI-SupportBundle.ps1

Purpose: Read-only packaging for the "Technical Blocker Mitigation — Defender for Identity" solution. When the sensor is on a supported OS with service reach, the DSA and audit policy are correct, and it still will not go healthy or stream (likely a Microsoft service issue), this assembles the bundle 'MDI Support Bundle - <case> - Escalate': the discovery snapshot, the DC audit-policy readiness output, the affected DC list, tenant context, and timestamps. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMDI-SupportBundle.ps1 -Case "MSCASE-12345" -AffectedDCs "DC01,DC02" Point it at the artifacts you already produced: .\TBMMDI-SupportBundle.ps1 -Case "MSCASE-12345" -Discovery .\MDI-blocker-discovery.json -AuditPolicy .\MDI-auditpolicy-DC01.json EXPECTED RESULTS A single folder 'MDI-Support-Bundle-<case>' with a manifest and copies of the read-only artifacts. ALSO run the MDI SENSOR's own log-collection ON the DC and add its output — that is the on-DC half Microsoft support asks for. .NOTES White-label. Read-only — collects existing read-only artifacts; changes nothing. Do not place the bundle in shared/unsecured storage; it contains DC/tenant identifiers. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Case, [string]$AffectedDCs = "", [string]$Discovery = ".\MDI-blocker-discovery.json", [string]$AuditPolicy = "", [string]$Report = ".\MDI-blocker-report.json")

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMDI-SupportBundle.ps1 -Case "MSCASE-12345" -AffectedDCs "DC01,DC02" Point it at the artifacts you already produced: .\TBMMDI-SupportBundle.ps1 -Case "MSCASE-12345" -Discovery .\MDI-blocker-discovery.json -AuditPolicy .\MDI-auditpolicy-DC01.json EXPECTED RESULTS A single folder 'MDI-Support-Bundle-<case>' with a manifest and copies of the read-only artifacts. ALSO run the MDI SENSOR's own log-collection ON the DC and add its output — that is the on-DC half Microsoft support asks for. .NOTES White-label. Read-only — collects existing read-only artifacts; changes nothing. Do not place the bundle in shared/unsecured storage; it contains DC/tenant identifiers. > [CmdletBinding()]

param([Parameter(Mandatory)][string]$Case, [string]$AffectedDCs = "", [string]$Discovery = ".\MDI-blocker-discovery.json", [string]$AuditPolicy = "", [string]$Report = ".\MDI-blocker-report.json")

TBMMDI-Validate.ps1

Purpose: Read-only validation for the "Technical Blocker Mitigation — Defender for Identity" solution. Runs the saved advanced-hunting logic 'MDI Sensor Blocker Check - Tenant - Diagnose' via Microsoft Graph Security to confirm identity signal for the affected DCs: logon events, directory events, and AD-query events (DSA). Returns a pass/fail per link. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMDI-Validate.ps1 -DeviceNameLike "DC01" Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS PASS/FAIL lines for logon/event signal and DSA query signal. FAIL points you to the matching Troubleshooting section. Cross-check sensor health in Settings -> Identities -> Sensors. .NOTES White-label. Read-only. Advanced-hunting queries only read telemetry; they change nothing. > [CmdletBinding()] param([string]$DeviceNameLike = "", [int]$LookbackHours = 24)

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMDI-Validate.ps1 -DeviceNameLike "DC01" Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS PASS/FAIL lines for logon/event signal and DSA query signal. FAIL points you to the matching Troubleshooting section. Cross-check sensor health in Settings -> Identities -> Sensors. .NOTES White-label. Read-only. Advanced-hunting queries only read telemetry; they change nothing. > [CmdletBinding()] param([string]$DeviceNameLike = "", [int]$LookbackHours = 24)